The Bank Is Open: Predicting NBA Over/Under Totals with Machine Learning

UE24CS352A - Machine Learning, Mini-Project | **Rohan M (PES1UG24CS381)**, **Sanupureddy Hemanth Kumar Reddy (PES1UG24CS418)** Code: https://github.com/rohancodexp/nba-overunder-ml | Final real-data experiment

Dataset	SBRO NBA odds archive, 2007-08 through 2022-23	**Games**	19,641 total; 19,592 usable
Split	Chronological: train through 2015-16; validation 2016-17; test 2017-18	**Features**	48 numeric + 60 team one-hot

1. Problem statement

Sportsbooks publish an Over/Under line representing the expected combined points of both teams in an NBA game. This project predicts the final total from information available before tip-off and evaluates predictions with Mean Squared Error (MSE), Mean Absolute Error (MAE), and Over/Under accuracy. The sportsbook closing line is kept as an evaluation reference rather than used as a model feature. At the standard -110 price, the break-even win rate is 52.38%. The work follows the problem framing of The Bank Is Open: AI in Sports Gambling.

2. Dataset

The real dataset was assembled from the Sportsbook Reviews Online (SBRO) NBA odds archive. Season files from 2007-08 through 2022-23 were processed while preserving the source rows and fields, including date, teams, quarter scores, final score, opening/closing totals, moneyline and second-half line. The resulting paired dataset contains 19,641 games, with 19,592 usable feature vectors. The evaluation is chronological rather than shuffled, so each game's features use only information available before that game.

3. Approach

Leakage-free features. Each team's last three games contribute points for/against, home/away status, rest, and opponent season-to-date scoring/defence. Current context includes rest days, a back-to-back flag, consecutive-away streak, games played in the previous seven days, season-to-date averages, and team one-hot encodings. Historical statistics are shifted so the current game cannot contribute to its own features.

Models. (i) Random Forest baseline; (ii) Collaborative Filtering using team interactions and recent league results with regularised factorisation; (iii) feed-forward Neural Network; (iv) an LSTM implemented in NumPy with back-propagation through time and gradient checking; and (v) an ensemble combining RF, NN and LSTM predictions. Hyper-parameters are selected on validation data and logged in `results/tuning_log.csv`.

4. Implementation overview

The implementation uses Python, NumPy, pandas, scikit-learn and matplotlib. `src/data_loader.py` reads season files; `src/features.py` constructs leakage-free features and fatigue context; `src/models/` contains the four base models; `src/tuning.py` performs validation-only tuning; `run_all.py` runs the experiment; and `demo.py` provides a prediction demo. Automated tests cover the pipeline and model behaviour, and final experiment artifacts are stored under `results/`.

5. Final real-data results

Random Forest	350.702	18.727	15.087	49.73%	-5.06%
CF-SVD	403.573	20.089	16.101	49.88%	-4.77%
Neural Network	348.797	18.676	14.941	49.19%	-6.10%
LSTM	344.240	18.554	14.861	49.73%	-5.06%

NN + LSTM + RF Ensemble	343.017	18.521	14.870	49.11%	-6.24%
Sportsbook line	320.303	—	—	—	—
Naive train mean	558.777	—	—	—	—

The final test set contains 1,311 games. The sportsbook closing line has lower test MSE than each learned model in this run. The ensemble has the lowest test MSE among the project's learned models. Over/Under accuracy is evaluated separately and pushes are excluded. These historical results do not establish a profitable wagering strategy.

6. Experiment design and validation

The final chronological split uses 11,532 training games, 1,309 validation games and 1,311 test games. The validation set is used for model selection, hyper-parameter tuning and neural-network early stopping; the final test set is held out for the reported comparison. The experiment also evaluates how much historical data is available at different training starts. Validation MSE was approximately 333.87 when training began in 2007-08, 334.34 from 2009-10, 339.97 from 2011-12, 337.96 from 2012-13, and 338.22 from 2013-14.

Model tuning. Random Forest, Neural Network and LSTM configurations were searched using validation data; CF-SVD used a parameter grid over factor count, history horizon and regularisation. The selected configurations and timing information are retained in the experiment log so that the final run can be reproduced and inspected.

7. Interpretation of the final results

Among the learned models, the ensemble records the lowest test MSE at 343.017, followed by LSTM at 344.240, Neural Network at 348.797, Random Forest at 350.702, and CF-SVD at 403.573. The naive training-mean baseline has a much larger test MSE of 558.777. The sportsbook closing line has a test MSE of 320.303, which is lower than the learned-model values in this experiment. Therefore, the results should be presented as a comparison of predictive error rather than as evidence that the system can beat the market.

Over/Under accuracy is also reported because the original problem concerns the side of the closing line. For the final test period, model accuracies are close to 50%, and the reported -110 ROI values are negative. These figures are descriptive measurements on the fixed historical test set; they should not be interpreted as a recommendation or as evidence of a deployable betting strategy.

8. Reproducibility and software structure

The repository is organised around a single end-to-end runner and modular source files. `src/config.py` stores configuration; `src/data_loader.py` loads the prepared season data; `src/data_synth.py` supports synthetic-data checks; `src/features.py` builds historical features; `src/pipeline.py` constructs the experiment data; `src/models/` contains RF, CF-SVD, NN and LSTM implementations; and `run_all.py` coordinates tuning, training, evaluation and output generation. The project also provides `demo.py` and an automated test suite.

The final repository keeps raw source spreadsheets outside version control and records the generated experiment artifacts needed to inspect the reported run. The README documents installation, data preparation, full and quick runs, demo usage, tests, output locations, methodology, and final results.

9. Conclusions and limitations

This project implements a leakage-aware, chronological pipeline for predicting NBA game totals from historical game and sportsbook data. The fatigue-related workload context and ensemble stage extend the baseline four-model pipeline. On the final test period, the learned ensemble reaches 343.017 MSE, while the sportsbook reference is 320.303 MSE. The experiment therefore demonstrates a complete modelling and evaluation workflow, but also shows that lower prediction error than a naive baseline does not imply that a model will outperform a market-derived line.

Limitations: the feature set is primarily score- and schedule-based and does not include player availability, injuries, line movement, or richer game-context variables. The reported test period is one historical season, so results can vary across periods. Future work can add player-level and market-movement information, use walk-forward validation over multiple seasons, and evaluate direct Over/Under classification alongside total-point regression.

Final reproducibility note: The full experiment produced 19,641 games and 19,592 usable feature rows. The final run reports 48 numeric features plus 60 team one-hot features. The repository README should be used as the operational guide for rerunning the project.